

Technical analysis of Notar-IA for a RealPage AI Engineer review

Scope of this analysis

This document explains the current Notar-IA system in technical terms for a technical reviewer. The goal is not to oversell it as a generic "AI app", but to map the actual implementation to the capabilities RealPage is looking for: multimodal document understanding, structured extraction, retrieval, orchestration, prompt engineering, exception handling, and production-minded architecture.

The version described here is the current effective product line we want to present:

- end-to-end document generation
- OCR and multimodal extraction
- strict structured outputs
- local domain RAG
- deterministic multi-step orchestration
- section-level generation
- validation and retries
- output traceability

This writeup does not position legacy feedback or auto-tune flows as the core product story.

Executive summary

Notar-IA is a Python-based document automation system for notarial deed generation. It takes heterogeneous legal inputs such as scanned identity documents, filing sheets, title certificates, prior deeds, tax clearances, powers, and corporate records, and turns them into a structured deed draft in DOCX and PDF.

From an engineering perspective, the key point is that this is not a single-prompt wrapper around an LLM. It is a staged pipeline implemented as an application-controlled orchestration layer on top of:

- FastAPI for the backend API
- Pydantic and pydantic-settings for typed contracts and config
- the OpenAI Python SDK for structured and multimodal inference
- pdfplumber and local PDF parsing for low-cost text extraction
- pdftoppm image rendering for scanned PDFs

- rank-bm25 for local retrieval
- python-docx, docxtpl, and LibreOffice-based conversion for output generation
- asyncio for concurrent document analysis
- React, Vite, and axios for the frontend

The most important architectural choice is that the control loop lives in the application, not in an unconstrained agent framework.

Deployment and infrastructure

This project also has a concrete deployment story, which is worth mentioning because it shows the system is not limited to a local development environment.

The deployment model is:

- VPS-based deployment
- deployed in practice on a DigitalOcean server
- Dockerized backend
- nginx reverse proxy
- static frontend hosting
- persistent artifact storage on the server filesystem

The repo contains the deployment assets for this:

- [DEPLOYMENT.md](#)
- [Dockerfile](#)
- [ops/deploy/docker-compose.yml](#)
- [ops/deploy/nginx/default.conf](#)

From an infrastructure perspective, this is useful because:

- the backend runs in a container
- uvicorn serves FastAPI inside the backend container
- the backend binds on loopback 127.0.0.1:8080
- nginx exposes the service, serves the frontend, and proxies /api/
- generated case outputs are persisted under /srv/notar-ia/data/outputs
- basic auth is enforced at the nginx layer

For a technical reviewer, this is a meaningful signal that the system was designed as an operable service, not just as a notebook or local script.

Stack and libraries

Backend

Core backend dependencies from pyproject.toml:

- fastapi
- uvicorn
- openai
- pydantic-settings
- pdfplumber
- pypdf
- pillow
- rank-bm25
- python-docx
- docxtpl
- reportlab
- httpx
- tenacity

Frontend

Core frontend dependencies from package.json:

- react
- react-dom
- vite
- axios

Architecture overview

At a high level, the system architecture looks like this:

- 1 React frontend uploads files using multipart requests.
- 2 FastAPI receives the request and writes files to a staging area.
- 3 The pipeline deduplicates files and identifies the filing-sheet candidate.
- 4 The system runs multimodal extraction for filing sheet, supports, CTL, and IDs.
- 5 Structured results are merged into a canonical universal context.
- 6 A local RAG layer retrieves the correct legal template for each act.
- 7 A binder generates the deed section by section using prompts plus retrieved legal text.
- 8 Validation and retry logic repair missing clauses or malformed structure.
- 9 The final result is rendered to DOCX and converted to PDF.
- 10 Debug artifacts are written for every stage.

The main orchestration entry point is [app/pipeline/orchestrator.py](#).

API and request flow

The main endpoint is the POST `/notaria-v63-universal` route in [app/api/routes.py](#).

What happens there:

- files are received as multipart form uploads
- staging folders are created
- uploaded files are persisted
- the system invokes `run_pipeline(...)`
- output artifacts are finalized and returned as a case response

This is already the right product shape for a production AI workflow: the LLM is not the API, the application is.

It is also worth noting that the frontend and backend are intentionally separated at deployment time:

- the frontend is built as static assets
- nginx serves the frontend directly
- nginx proxies API traffic to the backend container
- backend and frontend can be deployed independently

That is the kind of split that tends to age well in enterprise environments.

Pipeline design

1. Content-based deduplication

Before any inference, the system deduplicates uploaded files by SHA-256 content hash, not just by filename: [app/services/case_manager.py](#).

This matters because document workflows often include duplicate uploads, renamed copies, and repeated scans. Removing those early reduces:

- token cost
- redundant OCR
- retrieval noise
- latency

2. Filing-sheet candidate selection

The system does not blindly trust filenames such as `radicacion.pdf`. It first filters likely candidates and then scores them using semantic and structural signals: [app/pipeline/orchestrator.py](#), [app/pipeline/orchestrator.py](#).

This is a good example of application-level orchestration replacing agent guesswork. The model is used for extraction, but the selection policy stays deterministic.

3. Multimodal extraction strategy

The `OpenAIClient` implements a cost-aware extraction strategy in [app/services/openai_client.py](#):

- it first tries local PDF text extraction via `pdfplumber`
- if the extracted text is good enough, it uses plain text inference
- if not, it renders PDF pages to JPEGs with `pdftoppm`
- then it uses a multimodal model path

This is a strong production choice:

- cheaper than image-first for every document
- more stable for machine-readable PDFs
- still robust for scans

4. Strict structured outputs

Instead of asking the model for free-form prose, the pipeline uses typed extraction models defined in [app/schemas/extraction.py](#).

Examples include:

- `RadicacionExtraction`
- `SoporteExtraction`
- `CTLExtraction`
- `CedulaExtraction`

The OpenAI client uses structured completion parsing where possible, and falls back to JSON repair when necessary: [app/services/openai_client.py](#), [app/pipeline/json_repair.py](#).

This is one of the strongest parts of the system because it converts ambiguous model output into an application-owned contract.

5. Universal context construction

After extraction, the pipeline merges all document signals into a canonical context object. This context includes:

- current business act data
- parties and roles
- representation relationships
- property data
- title and registry information
- historical support signals

This merge happens in the orchestrator and supporting act engine logic: [app/pipeline/orchestrator.py](#), [app/pipeline/act_engine.py](#).

Conceptually, this is the system's shared state layer. If someone asked where the "memory" or "graph-like" part lives, this is the closest real answer.

Retrieval architecture

Local RAG design

The retrieval stack is implemented with:

- LocalRAG in [app/services/rag/local_rag.py](#)
- LocalRAGStore in [app/services/rag_store.py](#)
- KnowledgeRAG in [app/services/rag/knowledge_rag.py](#)

The corpus is local and curated:

- textual act templates
- .doc and .docx legal minuta files
- legal knowledge chunks for notarial rules

Retrieval uses rank - bm25 with domain-specific rescoring rather than a hosted vector database.

Why this is technically defensible

For this domain, local BM25 plus domain heuristics can be better than generic embeddings because:

- the corpus is relatively small and curated
- legal precision matters more than broad semantic fuzziness
- explainability matters
- retrieval mistakes are expensive
- operating cost should stay low

This is not "less advanced RAG". It is a domain-optimized retrieval strategy.

Template disambiguation

The system does not simply retrieve by act name. It derives act flags and retrieval profiles from contextual signals such as:

- property kind
- rurality
- baldío origin
- property-horizontal relevance

This logic is visible in:

- [app/pipeline/act_engine.py](#)
- [app/services/rag/local_rag.py](#)

This is important because it allows the system to avoid template drift between superficially similar acts.

Generation architecture

Binder model

The deed is not generated in one giant prompt. The pipeline creates "missions" or generation units for:

- header and filing sections
- opening sections
- act-specific sections
- insertions
- closing sections
- signatures

This section-level generation approach appears in the mission construction and mission execution logic in [app/pipeline/orchestrator.py](#), [app/pipeline/orchestrator.py](#).

From a systems perspective, this is one of the clearest examples of deterministic orchestration replacing a generic agent loop.

Prompt engineering

Prompting is specialized by task in [app/pipeline/prompts.py](#).

There are dedicated prompts for:

- filing-sheet extraction
- support document extraction
- CTL extraction
- ID extraction
- binder generation

This matters because prompt specialization is one of the main reasons the system behaves like a workflow engine instead of a single chat session.

Application-level guardrails

There is no external guardrail framework in the core loop. Instead, the system uses application-level guardrails:

- Pydantic schemas
- prompt constraints
- retrieval constraints
- required clause validation
- retry logic
- post-processing cleanup
- regression tests

This is a serious production pattern. It is often stronger than adding a generic guardrails library on top of an otherwise uncontrolled flow.

Validation and retry logic

One of the best engineering choices here is that the orchestrator does targeted retries.

If an act output:

- drops required clauses
- loses structure
- truncates a section

the system retries the missing portion rather than regenerating the entire document.

That logic is visible in the later sections of [app/pipeline/orchestrator.py](#).

This improves:

- cost
- stability
- debuggability

- consistency

Concurrency model

The pipeline uses `asyncio` and `asyncio.to_thread(...)` to parallelize I/O-heavy and model-bound steps, including:

- candidate analysis
- support extraction
- ID extraction
- mission execution

This appears repeatedly in [app/pipeline/orchestrator.py](#).

That is another sign that this was built as a pipeline system, not as an interactive notebook workflow.

Output generation

The final rendering path uses:

- `docxtempl` and document templating for Word output
- renderer logic in [app/services/rendering/docx_renderer.py](#)
- LibreOffice conversion for PDF

The final render step in the orchestrator is here: [app/pipeline/orchestrator.py](#).

This is useful for a technical reviewer because it shows that the system owns the artifact format, not just the text.

Observability and traceability

The system writes detailed artifacts for each run using `DebugDumper` in [app/pipeline/debug_dump.py](#).

Artifacts include:

- input manifests
- stage JSONs
- raw model outputs

- RAG hits
- mission definitions
- binder outputs
- final body text
- debug checklist

This is functionally similar to custom AI observability, even though it does not use LangSmith or OpenTelemetry.

Because artifacts are persisted on the VPS filesystem, that observability is operationally useful after deployment too. It gives you a simple but effective audit trail for:

- document ingestion
- extraction outputs
- retrieval decisions
- generation structure
- final case artifacts

Testing and quality

The repo includes regression-oriented tests, especially in:

- [tests/test_act_regressions.py](#)
- [tests/test_openai_client.py](#)

These tests are important because they encode domain-specific failure modes such as:

- wrong template selection
- clause truncation
- retrieval ambiguity
- act-specific structural regressions

That is stronger evidence of engineering maturity than a generic "we tested the API" claim.

Frontend architecture

The frontend is intentionally simple:

- React
- Vite
- axios

Its job is not to contain business logic but to:

- upload case files
- display pipeline results
- expose generated artifacts

The core UI entry point is [src/App.tsx](#).

For a technical reviewer, this is a healthy split: orchestration and correctness live in the backend.

Mapping to what RealPage is looking for

Multimodal and document understanding

Strong coverage:

- multimodal extraction
- scanned and machine-readable PDF support
- structured document understanding

Prompt engineering and guardrails

Strong coverage:

- task-specific prompts
- structured outputs
- schema-first extraction
- clause validation
- retry logic

Agentic or multi-step orchestration

Strong coverage in capability, but not via an agent framework.

The correct positioning is:

This system has agentic behavior at the workflow level, but the control loop is implemented in Python application code rather than delegated to an open-ended agent framework.

RAG and knowledge integration

Strong coverage:

- local retrieval
- domain-specific reranking
- legal knowledge chunks

Full-stack integration

Good coverage:

- React frontend
- FastAPI backend
- file uploads
- artifact rendering
- structured outputs returned to the client
- real VPS deployment with Docker and nginx

Monitoring and optimization

Good coverage:

- traceability artifacts
- cost-aware extraction path
- dedupe
- targeted retries
- regression tests

Honest gaps

The strongest version of this project still has clear boundaries:

- 1 No agent framework such as LangChain or OpenAI Agents SDK in the core path.
- 2 No hosted vector DB.
- 3 No MCP implementation.
- 4 No browser automation.
- 5 No standard observability stack like LangSmith or OpenTelemetry.
- 6 No Kubernetes story visible in this repo.

These are not fatal gaps. They are simply important to state precisely.

Best technical framing for RealPage

The strongest technical framing is:

I built a Python-based production workflow for legal document generation on top of FastAPI, Pydantic, the OpenAI SDK, local OCR and PDF parsing, domain-specific BM25 retrieval, and application-controlled orchestration. I intentionally kept the control loop deterministic because the hard part of this problem is not generic chat, it is reliable structure, retrieval precision, exception handling, and traceability.

And if the reviewer also values infrastructure maturity, the next sentence should be:

I also packaged and deployed it on a DigitalOcean VPS using Docker and nginx, with static frontend serving, reverse proxying, and persistent output storage for generated case artifacts.

Final conclusion

A technical reviewer should see Notar-IA as:

- a real workflow system
- an application-owned orchestration layer over model capabilities
- a strong example of structured AI engineering
- a good fit for document-heavy enterprise automation

The strongest closing thesis is:

This is not a prompt demo. It is a staged AI system with typed extraction, local retrieval, deterministic orchestration, validation, retries, and output traceability, designed for a document workflow where correctness and inspectability matter.